



CSI247: Data Structures

Lecture 02 – Review: Java Basics

Department of Computer Science
B. Gopolang (247-275)



Previous Lesson ...

- Course objectives, overview
- Topics to cover in the course
- Course material
- Course admin: schedule, pre-requisite, labs
- Assessments
- Useful tools
- Introduction to Data Structures

Today ...

Learning Objectives

In this lecture we will review:

- 1st year programming basics
 - Java program structure
 - Variables
 - Manipulating text
 - Control statements (selection and repetition)
 - Java defined Classes and Objects
 - API Documentation
 - Keyboard input
 - File IO

1. Java Program

- A collection of lines of code (to solve a specific problem)
- Structure
 - ```
public class className {
 public static void main(String[] args) {
 // statements here
 }
}
```
  - Program name matches **className**, with .java extension
  - `main()` – method
    - Point of entry during program execution

## 2. Variables

- Rem: Programs generally manipulate some data in memory
  - Data stored in a **variable**
    - Placeholder for data
    - Must be declared before use in Java – next slide
      - Given a unique **name** &
      - **data type**
        - A set of values with a set of permitted operations
          - a) Primitives
            - e.g. **int**: +, \*, ++, etc
          - b) Reference types (Classes)
            - e.g. **String**: charAt(), length(), indexOf(), etc

- Variable **declaration**
  - Allocate space for specified **data type** and give it a **name**
  - E.g. *int x;*
- Variable **initialization**
  - Gives a variable an initial value
  - E.g. *x = 5;* // *also an assignment in this case*
  - Use keyword **final** if it should not change
    - E.g.: *final int x = 5;*
- Memory diagram: Primitive vs Reference?

# 3. Operators

- Required to process the data

| Operator            | Examples                             |
|---------------------|--------------------------------------|
| Assignment          | <code>x = 17; x = x + 17; etc</code> |
| Arithmetic          | <code>+, -, *, /, etc</code>         |
| Relational          | <code>&gt;, &gt;=, &lt;, etc</code>  |
| Logical             | <code>&amp;&amp;, !, etc</code>      |
| Increment/decrement | <code>(++x, x--, etc)</code>         |

- Note:
  - Order of precedence **Important!**



## 4. Control structures

### a) Selection:

- if: *if* (  $4 > 7$  )  
 $x += 4;$
- if — else : *if* (  $4 > 7$  )  
 $x += 4;$   
*else*  
 $x += 8;$
- Multi-branch if
- Switch statement
- Nested if statements

### b) Repetition:

- *for* & *while*
  - Condition statement
  - Body of the loop
  - Update statement
- Nested loops
- Various possible loop errors
  - E.g.: Infinite loop, one too many/less iterations, missing update, etc

## 5. Strings

- Sometimes the data is text
- Java has a class called **String**
  - Has many methods (operations) for text manipulation

*e.g.*

```
String dept = "Computer Science";
```

```
System.out.println(dept.length() * 10);
```

```
int x = dept.indexOf("scie");
```

```
char ch = dept.charAt(dept.length());
```

```
String st = dept.substring(9) + 7;
```

## 6. Classes and Objects

- Java: an OOP language
  - Related methods grouped into a unit called a **class**
    - A blueprint used to create **objects**
  - Rem:

```
Scanner scan = new Scanner(System.in);
```

- 2 major class categories in Java
  - Pre-defined classes (next slide)
  - User-defined classes (later)

## 7. Pre-defined classes

- Java has a comprehensive list of pre-defined classes
  - Bundled within Java
  - Each is a collection of related **methods**
- Documentation of Java Library classes is accessible via:  
<https://docs.oracle.com/javase/8/docs/api/>
  - It lists all Java classes (and packages)
  - Each class:
    - List of methods & their descriptions
  - See next slide

## Java™ Platform, Standard Edition 8 API Specification

This document is the API specification for the Java™ Platform, Standard Edition.

See: Description

### Profiles

- compact1
- compact2
- compact3

### Packages

#### Package

**java.applet**

#### Description

Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.

**java.awt**

Contains all of the classes for creating user

## 8. Methods

- Written inside a class
- A sequence of related instructions
  - To achieve a specific task
  - Each has:
    - a) Name
    - b) Parameter list ( and a data type for each)
      - Optional: 0 or several parameters
    - c) Return type
      - void
      - primitive or class type

- Examples of methods from predefined classes
  - Scanner
    - **nextInt()**: takes no parameter, returns an int
    - **hasNext()**: takes no parameter, returns boolean
    - etc
  - String
    - **charAt()**: takes an int, returns a character
    - **length()**: takes no argument, returns an int
    - etc

## Invoking/calling a method

- You MUST provide:
  - The expected number of parameters
    - In the expected order
  - Data of the expected data type for each parameter
- If method returns value:
  - Store the returned value in a variable
  - Data types of the returned value and variable MUST match



## 9. Keyboard input

- Scanner class
- E.g.

```
import java.util.Scanner;
```

...

```
Scanner scan = new Scanner(System.in);
```

```
System.out.println("Enter number of pens needed: ");
```

```
int pens = scan.nextInt();
```

```
System.out.println("Enter unit price: ");
```

```
double price = scan.nextDouble();
```

...

## 10. Displaying output

- Various methods to use:
  - `System.out.print();`
    - Leaves cursor at end of displayed line
  - `System.out.println();`
    - Move cursor to beginning of next line
  - `System.out.printf()`
    - Uses formatting specifiers to format output
      - `%s`, `%5.2f`, `%c`, etc

# 11. File I/O

## a) Reading from a File

- Several options available
- E.g.: FileReader (or File) class & Scanner class

```
import java.util.Scanner;
```

```
import java.io.*;
```

...

```
FileReader fReader = new FileReader("data.txt");
```

```
Scanner scan = new Scanner(fReader);
```

...

## b) Writing to a File

- PrintWriter class

- E.g.

```
import java.io.*;
```

```
...
```

```
PrintWriter pw = new PrintWriter("out.txt");
pw.printf("Practicing File output");
pw.close();
```

## Note

- i. Input file **MUST** exist
  - If not: **FileNotFoundException**
- ii. Output file may not exist
  - If not there: Created
  - If there: Overwritten by default!
- iii. You **MUST** close files when done
  - Why?

## c) Catching file IO errors

- File operations are prone to various file-related issues
  - E.g. file not found
- Hence, catching File IO exceptions is vital
- Some File IO exception handling mechanisms in Java
  - throws statement
  - try-catch block

# Summary

- Review of some Java Programming basics
  - Java defined Classes and Objects
  - Java API documentation
  - Keyboard input
  - File IO
- 
- Next class
    - Methods

# Q & A



Thank you